

Es la unica forma que tienen los usuarios de interactuar con el sistema.
Coordina toda la interacción y lleva el registro de préstamos.

FichaDeLector

numero: int

cuit: string

nombreCompleto: string

esMoroso: bool

motivoDeRechazo: Symbol | nil

codigos: OrderedCollection<String>

FichaDeLector(deLector: Lector, conPrestamosVigentes: OrderedCollection<Prestamo>)

numero(): int

cuit(): string

nombreCompleto(): string

esMoroso(): Bool

puedeLlevar(): bool

motivoDeRechazo(): Symbol | nil

codigoEnPoder(): OrderedCollection<String>

cantidadEnPoder(): int

DatePort

now(): DateAndTime

Biblioteca

nombre: string

datePort: unDatePort

Biblioteca(nombre: unString, datePort: unDatePort)

agregarLector(l: Lector): Lector | nil

lectorNumero(n: int): Lector | nil

lectores(): OrderedCollection<Lector>

catalogar(m: MaterialCatalogado): MaterialCatalogado | nil

materialCodigo(c: String): MaterialCatalogado | nil

catalogo(): OrderedCollection<MaterialCatalogado>

materialesDe(unAutor: string): OrderedCollection<MaterialCatalogado>

prestarA(unCodigo: String, unNumero: int): Prestamo | nil

devolver(unCodigo: String): Prestamo | nil

estaPrestado(unCodigo: String): Bool

prestamoVigenteDe(unCodigo: String): Prestamo | nil

cantidadDePrestamosDe(unNumero: int): int

prestamos(): OrderedCollection<Prestamo>

prestamosDelMaterial(unCodigo: String): OrderedCollection<Prestamo>

prestamosVigentes(): OrderedCollection<Prestamo>

prestamosDe(unNumero: int): OrderedCollection<Prestamo>

prestamosVigentesDe(unNumero: int): OrderedCollection<Prestamo>

prestamosVencidos(): OrderedCollection<Prestamo>

esMoroso(unNumero: int): bool

lectoresMorosos(): OrderedCollection<Lector>

rehabilitar(unNumero: int): Bool

identificarSocio(unNumero: int): FichaDeLector | nil

identificarCuit(unCuit: string): FichaDeLector | nil

buscarLectoresPorApellido(unTexto: string): OrderedCollection<FichaDeLector>

Crea una biblioteca con catálogo y padrón vacíos, que usará ese reloj. El parámetro unString será el nombre de la biblioteca.
Retorna: instancia de Biblioteca

FakeDateAdapter

en(unDateAndTime: DateAndTime): FakeDateAdapter

enDiaMesAnio(dia: int, mes: int, anio: int): FakeDateAdapter

now(): DateAndTime

forzarFecha(unDateAndTime: DateAndTime): self

forzarDiaMesAnio(dia: int, mes: int, anio: int): self

forzarDiaMesAnioHoraMinuto(dia: int, mes: int, anio: int, hora: int, minuto: int): self

avanzarDias(unaCantidad: int): self

avanzarHoras(unaCantidadHoras: int): self

Persona

nombre: string

apellido: string

cuit: string

nombre(): string

apellido(): string

cuit(): string

Prestamo

material: MaterialCatalogado

lector: Lector

inicio: DateAndTime

vencimiento: DateAndTime

devolucion: DateAndTime | nil

Prestamo(material: MaterialCatalogado, lector: Lector, instante: unInstante)

material(): MaterialCatalogado

lector(): Lector

inicio(): DateAndTime

vencimiento(): DateAndTime

devolucion(): DateAndTime | nil

plazoEnDias(): int

registrarDevolucion(unInstante: DateAndTime): self

estaVigente(): Boolean

estaCerrado(): Boolean

dias(): int | nil

diasTranscurridosHasta(unInstante: DateAndTime): int

estaVencidoA(unInstante: DateAndTime): Boolean

excedioPlazo(): Boolean

descripcion(): string

pregunta por un prestamo que ya volvió, y se usa al devolver para decidir la mora. Un préstamo cerrado nunca está vencido, y uno abierto nunca excedió el plazo todavía.

pregunta por un préstamo que sigue afuera, y se usa para listar los vencidos hoy. Un préstamo cerrado nunca está vencido, y uno abierto nunca excedió el plazo todavía.

MaterialCatalogado

titulo: string

autor: OrderedCollection<string>

editorial: string

anioDeEdicion: int

paginas: int

titulo(): string

autores(): OrderedCollection<string>

autorPrincipal(): string | nil

esDeAutor(unNombre: string): bool

editorial(): string

anioDeEdicion(): int

paginas(): int

descripcion(): string

codigo(): string

plazoEnDias(): int

tipo(): string

tipoDeIdentificador(): string

Lector

numero: int

esMoroso: bool = false

Lector(numero: int, cuit: string, nombre: string, apellido: string)

numero(): int

cuit(): string

nombre(): string

apellido(): string

nombreCompleto(): string

apellidoContiene(unTexto: string): bool

esMoroso(): bool

puedeLlevar(): bool

motivoDeRechazo(): Symbol | nil

marcarMoroso(): self

rehabilitar(): self